PYTHON DATA APPS

Dash in 30 Minutes

From a Plotly chart to an interactive app, and what happens behind the scenes

Demos: `demos/` in this repo · start them all with `python demos/run_all.py`

Agenda

1. Plotly vs Dash: why a server 5 min
2. Building a page: layout, charts, widgets 7 min
3. Callbacks, and clicking one chart to update another 8 min
4. Letting Copilot write Dash for you 4 min
5. Showcase, and other tools 5 min

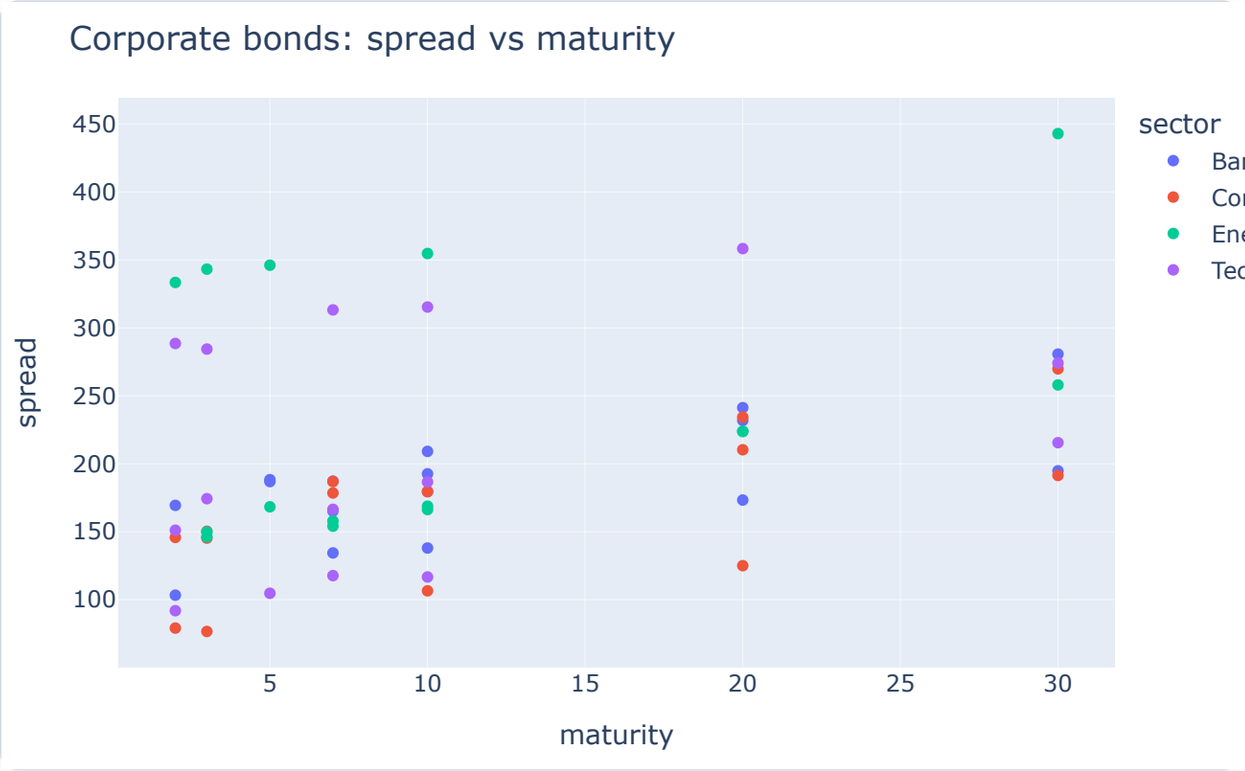
Plotly: interactive charts, no server

```
import plotly.express as px
from data import bonds

fig = px.scatter(bonds, x="maturity", y="spread",
                 color="sector", hover_name="issuer")
fig.write_html("plotly_only.html")
```

Needs Python again

- recomputation
- call a model, run a pricer
- data too big to ship into one HTML file / query a database

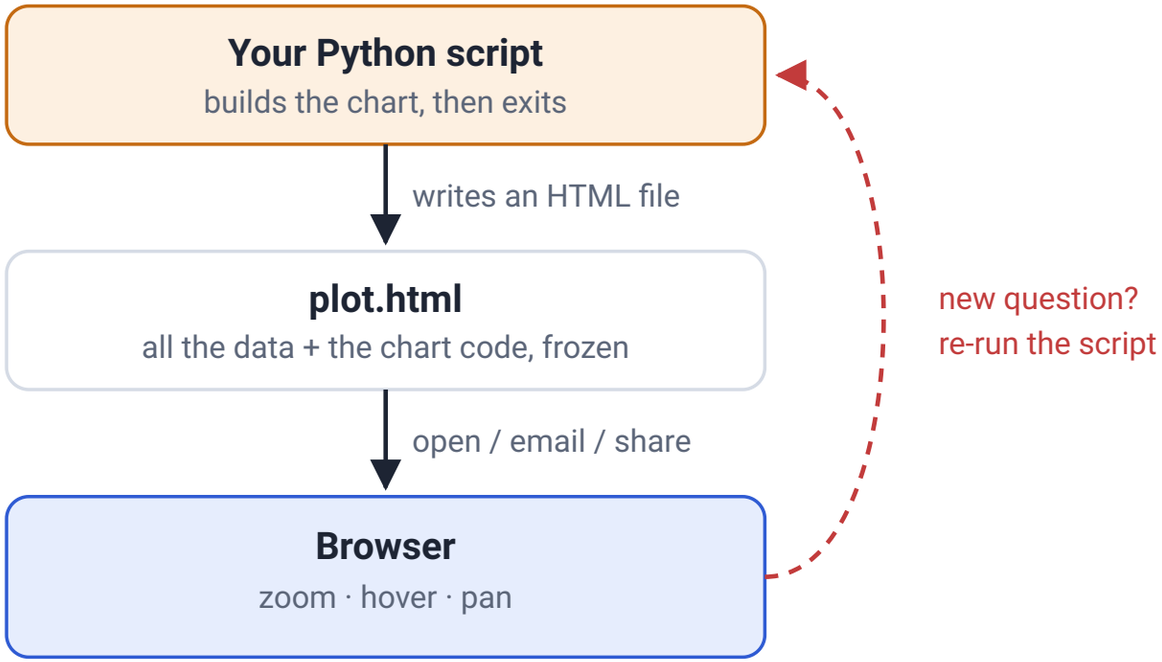


Works in the browser: zoom, hover, legend.

Plotly has no back end. Once the HTML is written, nothing can ask Python a new question.

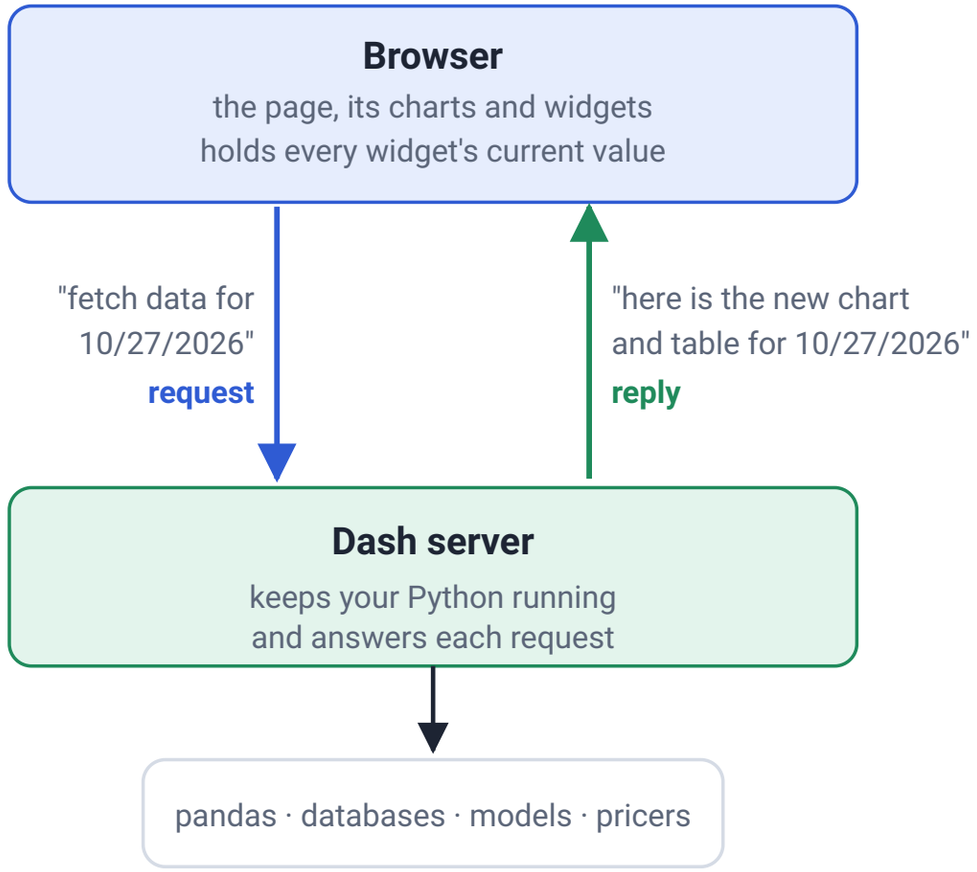
Plotly vs Dash

Plotly alone: one way, once



No server: nothing is listening after the file is written.

Dash: a conversation



The smallest Dash app

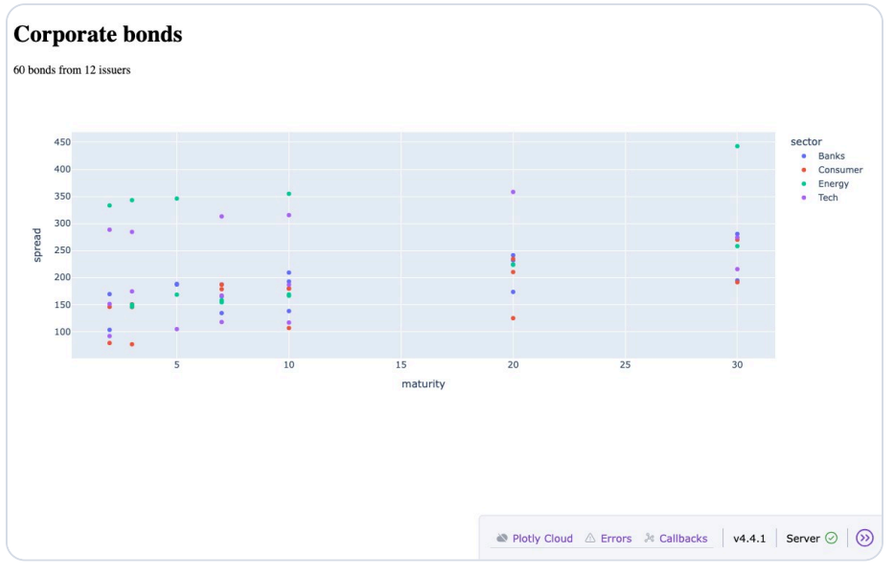
```python
from dash import Dash, html, dcc
import plotly.express as px
from data import bonds

app = Dash(__name__)

fig = px.scatter(bonds, x="maturity", y="spread", color="sector")

app.layout = html.Div([
    html.H1("Corporate bonds"),
    html.P(f"{len(bonds)} bonds"),
    dcc.Graph(id="chart", figure=fig),  # id: find it later
])

if __name__ == "__main__":
    app.run(debug=True)     # http://127.0.0.1:8050
```



demos/02_hello_dash.py

Components nest like boxes: the order in the code is the order on the page. An `id` lets the app find a component later, to read it or update it.

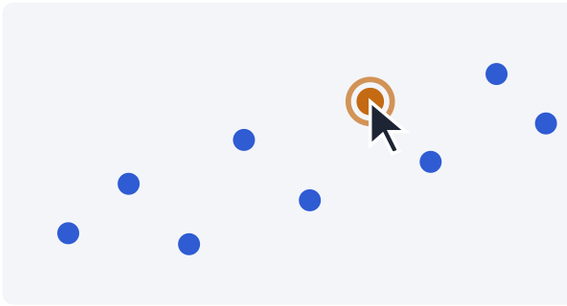
Plotly charts inside Dash

`dcc.Graph` shows any Plotly chart: the same code you'd write in a notebook.

```
dcc.Graph(
    id="chart",
    figure=px.line(df, x="date",
                   y="spread"),
)
```

A chart also tells the app what the user does to it.

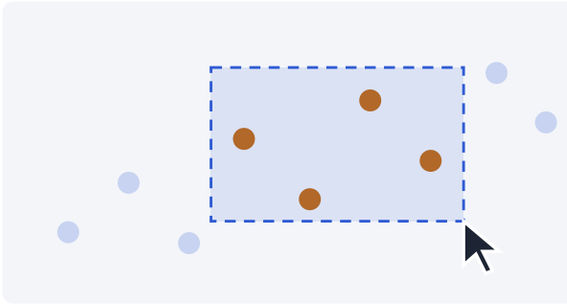
Later we'll show how to use `clickData` to update another chart when a point is clicked.



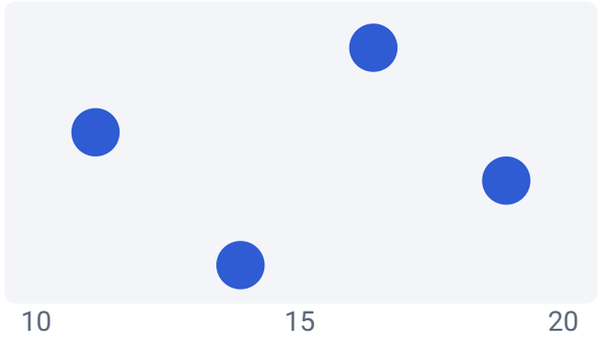
Click a point: the app learns which point
`clickData`



Hover a point: the app learns which point is under the mouse
`hoverData`

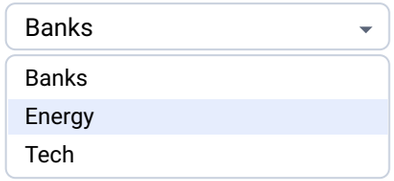


Draw a box or lasso: the app learns which points are inside
`selectedData`



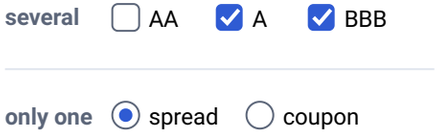
Zoom or pan: the app learns the new visible range
`relayoutData`

Widgets you get for free



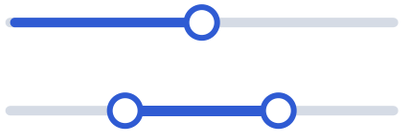
Pick from a list: the app gets the chosen item (or several)

`dcc.Dropdown`



Checklist: tick several. Radio: exactly one. The app gets what is selected

`dcc.Checklist` · `dcc.RadioItems`



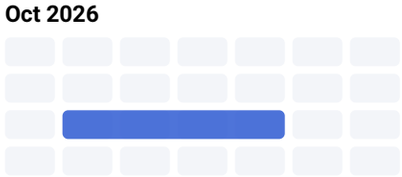
Drag a handle: the app gets the number, or the range

`dcc.Slider` · `dcc.RangeSlider`



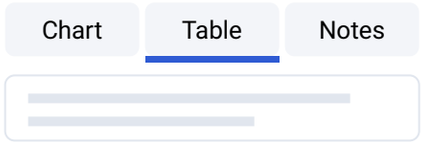
Type text or a number: the app gets what was typed

`dcc.Input`



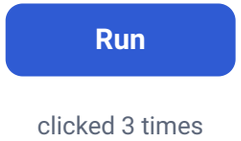
Pick a date or a range: the app gets the dates

`dcc.DatePickerSingle` · `DatePickerRange`



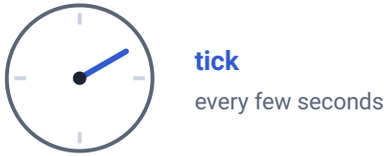
Switch tabs: the app gets which tab is open

`dcc.Tabs`



Press a button: the app gets how many times it was clicked

`html.Button`



A timer: the app gets a tick every few seconds, for live updates

`dcc.Interval`



Keep data between steps; let files in and out

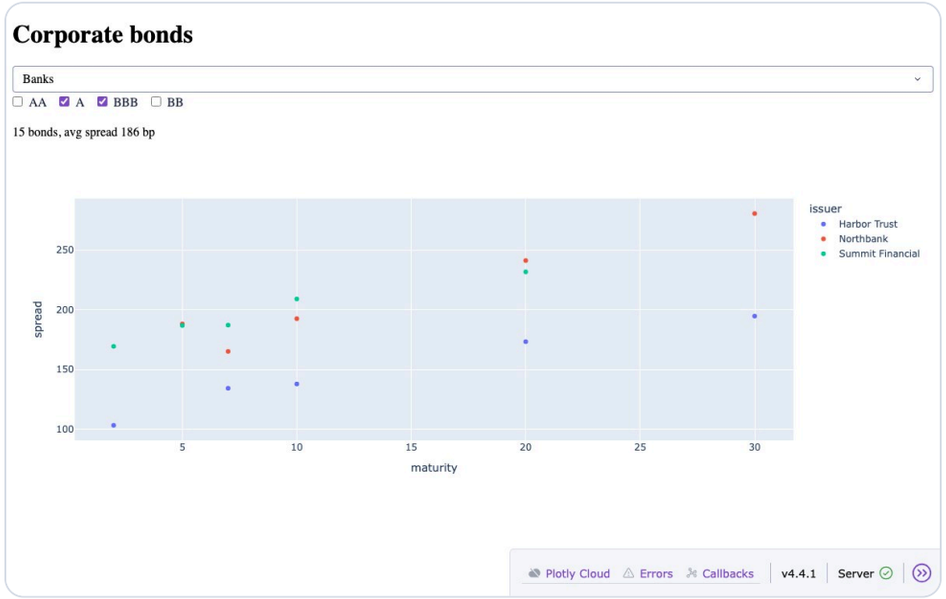
`dcc.Store` · `dcc.Upload` · `dcc.Download`

Add-on libraries: `dash-ag-grid` (sortable, filterable tables) · `dash-bootstrap-components` and `dash-mantine-components` (themes, layouts, more widgets)

A callback: "when this changes, run that"

```
app.layout = html.Div([
    html.H1("Corporate bonds"),
    dcc.Dropdown(sectors, "Banks", id="sector"),
    dcc.Checklist(ratings, ["A", "BBB"], id="rating"),
    html.P(id="summary"),
    dcc.Graph(id="chart"),
])

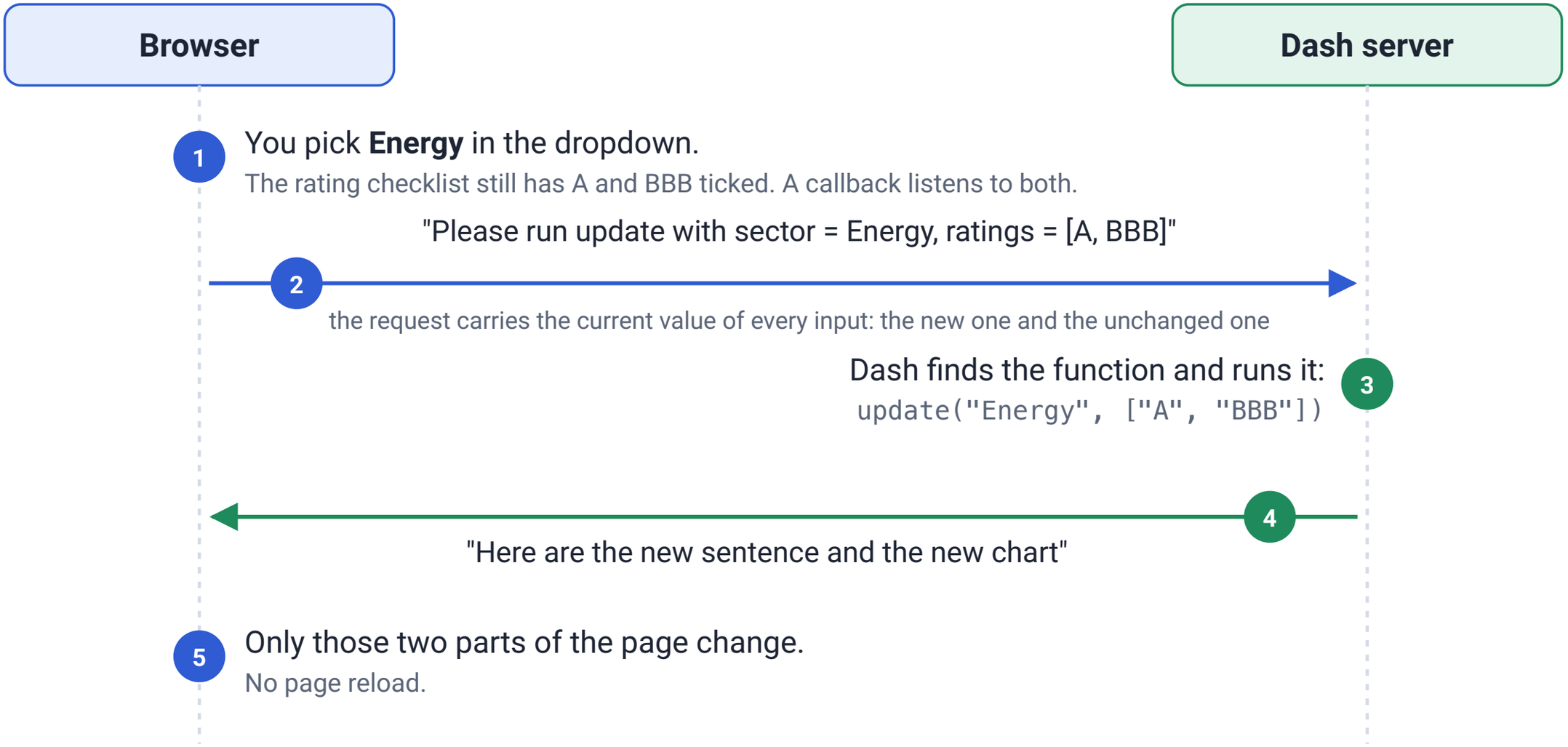
@callback(Output("summary", "children"),
          Output("chart", "figure"),
          Input("sector", "value"),
          Input("rating", "value"))
def update(sector, ratings):
    d = bonds[(bonds.sector == sector)
              & bonds.rating.isin(ratings)]
    text = f"{len(d)} bonds, avg {d.spread.mean():.0f} bp"
    fig = px.scatter(d, x="maturity", y="spread")
    return text, fig
```



Inputs → arguments, in order. The first `Input` arrives as the first argument (`sector`), the second as the second (`ratings`).

Return values → Outputs, in order. `text` fills the first `Output`, `fig` the second.

How the browser hands the work back to Python



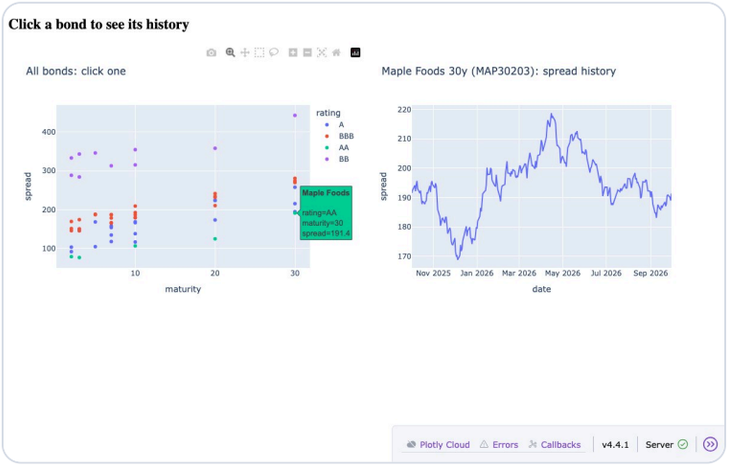
3 · CLICK ONE CHART, UPDATE ANOTHER

Click a bond, see its history

```
overview = px.scatter(bonds, x="maturity", y="spread",
                      color="rating", hover_name="issuer",
                      custom_data=["cusip"])   # rides along with each point

app.layout = html.Div([
    dcc.Graph(id="overview", figure=overview),
    dcc.Graph(id="detail"),
])

@callback(Output("detail", "figure"),
          Input("overview", "clickData"))
def show_history(click):
    if click is None:                       # page load: nothing clicked
        cusip = bonds.cusip.iloc[0]
    else:
        cusip = click["points"][0]["customdata"][0]
    h = history[history.cusip == cusip]
    return px.line(h, x="date", y="spread", title=cusip)
```



demos/05_click_detail.py

4 · VS CODE + COPILOT

Let Copilot write the Dash code

Describe what you want: at a high level, or in more detail if you know the details.

High level

Build a Dash app for the bonds data: filter by sector, show spread vs maturity, and let me click a bond to see its history.

More detailed

Add a sector dropdown (id `sector`, default Banks). When it changes, filter the scatter and show the count and average spread under it. When I click a point, use `custom_data` to get the cusip and plot its history from `history` in a second chart.

Tip: write your rules down once (a Copilot instructions file or a skill): which Dash version to use, where the data comes from, your naming conventions. Copilot then follows them on every request.

5 · SHOWCASE

Showcase: a real dashboard

Switching to the dashboard now.

5 · OTHER TOOLS

Dash and other Python app tools

tool	best fit	front-end model	server
Dash	production dashboards and apps with clear interactivity; Plotly charts	the page is described in Python; the browser holds widget values and asks the server when an input changes	Python web server; each request is answered on its own, with no per-user memory
Streamlit	quick internal tools; turning a data script into an app fast	the whole script re-runs top to bottom on every interaction; widgets simply return values	Python server with a live connection per user; per-user state in `st.session_state`
Panel / Bokeh	notebook-to-app; scientific or large data; mixing plotting libraries	Python objects kept in sync with their browser copies; reactive functions	Bokeh server with a live connection per user; can also run inside Jupyter
Jupyter widgets	interactive exploration inside a notebook	widgets in the notebook, kept in sync with Python objects	the notebook's Python kernel; Voilà turns a notebook into a shareable page
Shiny for Python	apps with many linked inputs and outputs	reactive: outputs re-compute automatically when the inputs they use change	Python server with a live connection per user
Gradio	demos of ML models: inputs in, prediction out	a function wrapped in ready-made input and output components	Python server; easy to share or host on Hugging Face